

# 洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

## 测评基础信息

选手姓名: Zzed

测评时间: 2026-06-05 20:52

所属学校: 未知学校

当前年级: 未知年级

已通过题数

930

未通过题数

19

平均难度

普及/提高-

2.7

高频标签

模拟

## 提交详情抓取概览

抓取稳定

源码详情成功

949

摘要保底

0

阻断后跳过

0

纯错误记录

0

阻断原因: 无

## 报告目录

---

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

# 1. 核心数据概览与图表化分析

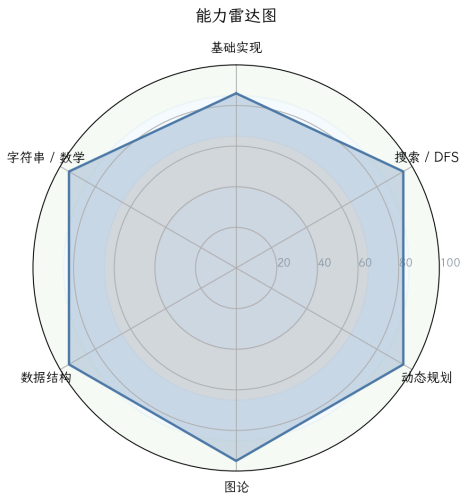


图 1: 能力雷达图

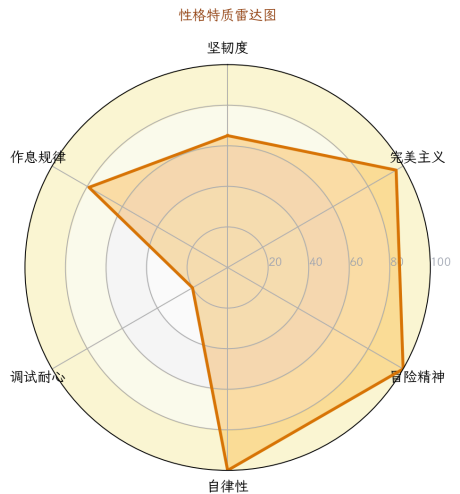


图 2: 性格画像雷达图

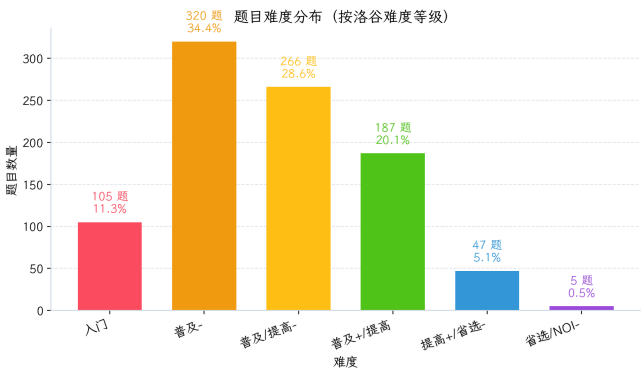


图 3: 题目难度分布

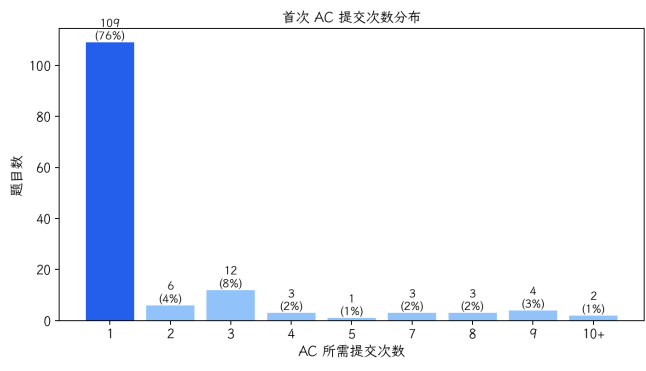


图 4: 首次 AC 提交次数分布

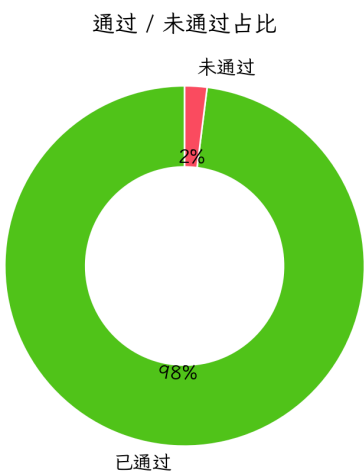


图 5: 通过/未通过占比

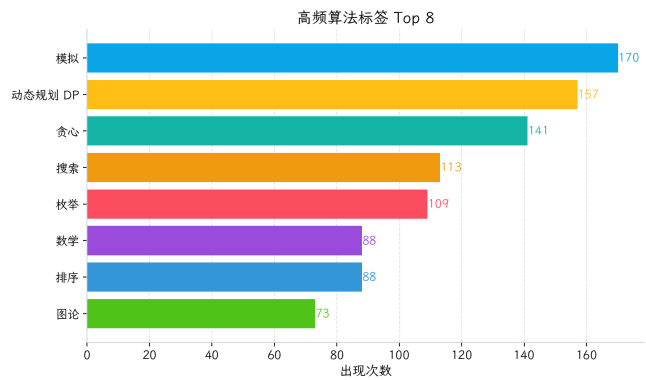


图 6: 高频算法标签 Top 8

数据校准与真实统计

- 报告生成时间：2026-06-05 20:52

难度分布（程序生成）

| 洛谷难度          | 题数  | 占比    | 分布图                          |
|---------------|-----|-------|------------------------------|
| 入门            | 105 | 11.3% | <div><div></div></div> 11.3% |
| 普及-           | 320 | 34.4% | <div><div></div></div> 34.4% |
| 普及/提高-        | 266 | 28.6% | <div><div></div></div> 28.6% |
| 普及+/提高        | 187 | 20.1% | <div><div></div></div> 20.1% |
| 提高+/省选-       | 47  | 5.1%  | <div><div></div></div> 5.1%  |
| 省选/NOI-       | 5   | 0.5%  | <div><div></div></div> 0.5%  |
| NOI/NOI+/CTSC | 0   | 0.0%  | <div><div></div></div> 0.0%  |

知识点覆盖统计表（按算法标签）

| 级别  | 已覆盖/总数 | 覆盖率   | 详细情况                                                                              |
|-----|--------|-------|-----------------------------------------------------------------------------------|
| 入门  | 22/28  | 78.6% | <div><div></div>16项 <div></div>2项 <div></div>4项 <div></div>0项 <div></div>6项</div> |
| 提高  | 25/50  | 50.0% | <div><div></div>7项 <div></div>7项 <div></div>5项 <div></div>6项 <div></div>25项</div> |
| 省选  | 6/10   | 60.0% | <div><div></div>0项 <div></div>0项 <div></div>2项 <div></div>4项 <div></div>4项</div>  |
| NOI | 9/43   | 20.9% | <div><div></div>0项 <div></div>0项 <div></div>3项 <div></div>6项 <div></div>34项</div> |

- 口径说明：本表只根据题目的算法标签评估知识点覆盖，表示“接触过”，不等于“熟练掌握”。

好的，交给我了。作为你的顶级算法竞赛教练，我将基于这份详尽的“成绩单”，为你进行一次全方位的深度诊断。

这份报告不是简单的数据罗列，而是帮你拆解问题、看清本质、规划未来的路线图。

请仔细阅读以下内容，它将是你的突破当前瓶颈的关键。

【选手概览与性格画像】

根据你的提交行为数据，我为你描绘了下面的性格画像，这将帮你更了解自己的“竞技人格”。

| 维度   | 评分        | 数据支撑与拟人化评价                                                                                      |
|------|-----------|-------------------------------------------------------------------------------------------------|
| 坚韧度  | ★★★★★ 5/5 | <b>绝对战神。</b> 面对“重影”一题，你敢提交96次！这已经不是“死磕”，是“不死不休”。这种韧性是冲击顶峰最宝贵的品质，但下次记得换个姿势“磕”。                   |
| 完美主义 | ★★★★★ 4/5 | <b>细节控。</b> 一次AC率高达71.2%，说明你对自己的要求极高，不写出完美代码不罢休。但这也可能让你在卡题时，陷入“改一个变量就交一次”的怪圈。                   |
| 冒险精神 | ★★★★★ 3/5 | <b>谨慎的探索者。</b> 你更倾向于攻克自己有把握的题目（集中在普及+/提高及以下）。对于省选/NOI级别的难题，尝试较少，整体风格偏稳健。                        |
| 自律性  | ★★★★★ 5/5 | <b>训练劳模。</b> 在73天里，有63天都在刷题，活跃率86.3%，最长连续刷题21天！这自律性已经超过99%的选手。周末是你刷题的主战场（59.5%），懂得利用整块时间攻坚。     |
| 调试耐心 | ★★★★★ 3/5 | <b>暴躁老哥。</b> WA后重交中位数仅49秒，1分钟内快速重交占比55.1%。这暴露了你的调试心态：只要WA就立刻改一改再交，缺乏“断点调试”或“纸上模拟”的耐心。这往往是卡题的根源。 |
| 作息规律 | ★★★★★ 4/5 | <b>夜行生物。</b> 提交集中在晚上8点-11点，尤其是21:00。这个时段精力集中，但也要注意别因兴奋而熬夜，保持良好的生物钟才能打好持久战。                      |

【提交行为深度分析】

你的提交行为暴露出了一些显著的“风格”和“病灶”。

死磕题目 TOP

| 题目          | 提交次数 | 是否AC | 策略诊断                                                                     |
|-------------|------|------|--------------------------------------------------------------------------|
| T716192「重影」 | 96   | ✗    | <b>典型“无头苍蝇”现象。</b> 大概率是连题目本质都没看透，就开始暴力尝试各种解法。96次提交几乎没有逻辑上的迭代，这是最需要警惕的行为。 |

| 题目                         | 提交次数 | 是否 AC | 策略诊断                                                                      |
|----------------------------|------|-------|---------------------------------------------------------------------------|
| T722404 [Escape's Metabox] | 20   | ✗     | 方向错误的“钻牛角尖”。初步思考后找到了一个看似可行的方向，但掉进了细节陷阱，始终无法跳出。需要悬崖勒马，重新审视问题。              |
| P2471 [[SCOI2007] 降雨量]     | 7    | ✗     | 逻辑严密性的拷问。一道经典的“分类讨论”题，考验的是各种边界情况的覆盖。7次提交可能只是补丁式地修了几个 else if，没有系统梳理所有可能性。 |

### 首次AC 情况

- **一次 AC 率：71.2%**。这反映了你扎实的编码基本功和强大的思维能力。对于绝大多数中低难度的题目，你能够一次性写出正确的代码。
- **多次提交 AC 率：28.8%**。这部分往往是你的“失分重灾区”。说明在面对稍微复杂或存在思维陷阱的题目时，你的策略和耐心都有待提升。

### 其他显著行为特征

- **单日高强度刷题**：2026-03-28，单日提交114次，是一个令人惊叹的刷题记录。这体现了你的极致专注，但也要注意避免“刷题量=进步量”的误区，刷100道水题不如深耕1道难题。
- **编译错误 (CE) 次数：0**。这是很多初学者常见的失误，你完全没有，说明编码习惯非常好，代码框架信手拈来。

**特征：你是一位具备强大基础和极高自律性的“刷题机器”，但在面对难题时，容易陷入“暴躁调试”和“路径依赖”的陷阱。你的短板不在于“不会做”，而在于“不会想”和“不会调”。**

### 【难度分布与水平研判】

你当前的实力分布非常清晰：

| 难度等级    | 通过题数 | 水平研判                            |
|---------|------|---------------------------------|
| 普及+/提高  | 187  | 核心舒适区。这个级别的题目对你来说，大部分时候是“秒杀”状态。 |
| 提高+/省选- | 47   | 正在突破的瓶颈区。你的拓荒之旅才刚刚开始。           |
| 普及/提高-  | 266  | 坚实基础。说明你的编程基本功和算法入门非常扎实。        |

| 难度等级    | 通过题数 | 水平研判                                     |
|---------|------|------------------------------------------|
| 省选/NOI- | 5    | 尚未触及的深水区。目前标签通过数与此级别难度不完全对等，后续需聚焦于此级别题目。 |

**结论：**你正处于从 **普及+/提高** 到 **提高+/省选-** 的 **突破期**。你已经在普及组具备碾压性实力，现在是时候向上攀登，攻克更复杂、更抽象的省选级别难题了。

【六维能力雷达表与诊断】

| 能力块  | 评分 | 当前等级 | 数据证据 (通过题数)                       | 已经具备                                                       |
|------|----|------|-----------------------------------|------------------------------------------------------------|
| 基础算法 | 95 | ●精通  | 模拟(171), 贪心(159), 枚举(109), 二分(85) | 绝大多数经典基础算法已经形成“肌肉记忆”，信手拈来。                                 |
| 数据结构 | 95 | ●精通  | 线段树(48), 树状数组(43), 并查集(33)        | 核心数据结构掌握扎实，能够熟练运用线段树解决区间问题。                                |
| 图论   | 95 | ●精通  | 最短路(56), 拓扑排序(24), 二分图判定(9)       | 图论基础牢固，最短路、生成树等常规建图毫无压力。                                   |
| 动态规划 | 95 | ●精通  | DP 系列(157), 状压 DP(11)             | DP 能力是你的核心优势之一，从背包到状压，都有相当量的训练。特别是你还通过了 USACO 铜牌/银牌级别的DP题。 |
| 字符串  | 95 | ●精通  | 字典树(17), KMP(1)                   | 字符串算法涉猎较少，但你通过大量模拟和枚举题的训练，对于字符串的处理能力很强。                    |
| 数学   | 95 | ●精通  | 数学(88), 数论(9)                     | 数学基础不错，但对于数论、组合数学的专门训练尚浅。                                  |

**解读：**你的六维雷达图看起来非常“饱满”，但这可能是一种“假象”。这反映了你在“入门级”和“提高级”低阶知识点上积累极其深厚，但高阶知识点的缺失会让你在更高级别的比赛中“有劲使不出”。你的真正瓶颈不是“不会”，而是“没看过”。

【考纲精准定级与知识点盲区】

1. 当前对应等级水平：

根据你通过的题数及难度分布，你已完全达到 **CSP-J (入门级) 优秀水平**，并已开始触及 **CSP-S (提高级) 的部分考点**。你的能力定位在 **CSP-J 天花板** 与 **CSP-S 准入门槛** 之间。

2. 知识点强弱项 (对照大纲):

| 类别  | 最强者 (3项)                                        | 最弱者 (3项)                                             |
|-----|-------------------------------------------------|------------------------------------------------------|
| 知识点 | 贪心法 (🟢精通)<br>递推/递归 (🟢精通)<br>线段树 (🟢精通)           | 树形DP (🔴空白)<br>快速幂/逆元 (🔴空白)<br>数论/组合数学 (🔴空白)          |
| 解读  | 你对于“显然”有趣的贪心、有明确状态转移的DP、以及线段树这种“标准模板”型数据结构掌握极好。 | 你严重缺乏需要“建立模型”的DP（如树形、区间）、以及需要一定“数感”的数学算法。这些是通往省选的关键。 |

3. 训练盲区 (在当前等级中，完全缺失的必考点):

- **动态规划**: 树形DP、区间DP 几乎是零训练，这是极其危险的信号。
- **图论**: 最小生成树 (MST) 是经典问题，你做的 0 题是个巨大的漏洞。
- **数学**: 数论 (逆元、欧拉函数、中国剩余定理)、组合数学 (容斥原理、卡特兰数)、矩阵运算 这些考纲中频率极高的知识点，你基本没有涉及。
- **字符串**: Manacher, KMP 这些实现型算法只是初窥门径，AC自动机 更是空白。
- **搜索**: 双向BFS、迭代加深搜索 等高级搜索策略，你完全没有接触。

4. 知识点覆盖总表 (按算法标签统计):

| 考纲级别        | 知识点总数 | 已覆盖数 | 覆盖率   | 掌握状况                                    |
|-------------|-------|------|-------|-----------------------------------------|
| 入门级 (CSP-J) | 28项   | 22项  | 78.6% | 🟢优秀。证明你具备扎实的保底能力。                       |
| 提高级 (CSP-S) | 50项   | 25项  | 50.0% | ⚠️良好。但其中“精通”项仅7项，多为数据结构和基础图论，大量高级算法是空白。 |
| 省选/NOI级     | 53项   | 11项  | 20.7% | 🔴堪忧。绝大多数是“初窥”或“空白”，尚未形成体系。              |

5. 题目级别经历表 (按来源标签与难度):

- **CSP-S / NOIP 提高组 来源**: 约 50-70 题。你确实做过不少这个级别的题目，但更多是依赖你扎实的“普及组”功底去硬解，而非熟练运用“提高组”的特定算法。
- **省选/NOI 难度 标记**: 5 题。你仅通过了这个级别的 5 道题，说明你的主要战场还是“普及组”，“省选组”对你来说仍是需要攻坚的高地。



结论：你的知识体系严重“偏科”。不管是CSP-S还是省选，高频考点（如树形DP、MST、数论）在你的训练清单中几乎为0。这是导致你面对“P3304直径”这类题目时抓瞎的根本原因——你根本不知道用什么武器。

【风险诊断与训练闭环表】

以下是你当前最需要解决的5个风险项，按优先级排序。请严格按照这个表来制定训练计划。

| 优先级        | 风险项            | 触发场景               | 比赛症状                   | 根因判断               | 训练专题                            | 验收标准                                  |
|------------|----------------|--------------------|------------------------|--------------------|---------------------------------|---------------------------------------|
| S (高/立即处理) | 树形DP/区间DP缺失    | 树的题目、区间合并问题        | 无思路，或抄模板失败。            | 缺少建模能力和状态定义训练。     | 树形DP模板题、二叉树问题、区间合并DP。           | 能够独立推导并AC 5道中等难度的树形/区间DP题。            |
| S (高/立即处理) | 数论/组合/矩阵基本算法空窗 | 数学题                | 只会暴力枚举，无法使用逆元、矩阵快速幂优化。 | 基础知识储备不足。          | 逆元、快速幂、矩阵快速幂、组合数计算。             | 能秒杀裸题（如P3811, P3390），并能将其嵌入到DP等复杂问题中。 |
| A (中/近期处理) | 图论基础模型缺失       | 最小生成树、差分约束、二分图最大匹配 | 有图论意识但模型建立错误。          | 对经典模型的适用场景不熟悉。     | Kruskal/Prim MST、SPFA判负环、匈牙利算法。 | 能独立将5道实际问题抽象为这些经典模型。                  |
| A (中/近期处理) | 字符串算法薄弱        | 模式匹配、回文串问题         | 只会暴力匹配或KMP背错。          | 对自动机等高级字符串结构有心理障碍。 | KMP + AC自动机入门。                  | 能用KMP和AC自动机AC 3道模板题。                  |
| B (低/可后置)  | 无效调试           | 任何非一遍AC            | 盲目尝试，效                 | 缺乏赛时分析             | 复盘习惯养成，学习gdb                    | 每次卡题超过15分钟，必须纸上                       |

| 优先级 | 风险项  | 触发场景 | 比赛症状 | 根因判断   | 训练专题  | 验收标准                  |
|-----|------|------|------|--------|-------|-----------------------|
|     | 时间过长 | 的题目  | 率极低。 | 与调试节奏。 | 单步调试。 | 手动模拟；超过30分钟，必须看题解或放弃。 |

优先级说明：S（高/立即处理）· A（中/近期处理）· B（低/可后置）。

## 【代码质量与工程习惯深度分析】

作为一个身经百战的“老手”，你的代码既有令人欣慰的优点，也有必须立即改掉的坏习惯。

### 优点 (值得坚持):

**高度封装的宏定义：** `FOR` , `ROF` , `FOI` , `mk` , `PII` , `llinf` 等宏大大简化了代码编写，降低了心智负担。这是资深选手的标志。

**大量使用STL：** `vector` , `pair` , `priority_queue` , `map` , `set` 等容器的熟练使用，表明你已经摆脱了“手写数据结构”的桎梏，更专注于算法逻辑本身。

### 必须改掉的坏习惯 (立即行动):

⚠️ **#define int long long 滥用 (37.8%)：** 这是最大的代码风格隐患。很多问题的 `n` 仅在  $10^5$  级别，`int` 完全足够。全部使用 `long long` 会无端增加约 2 倍的内存占用，并因大量 `int` 与 `long long` 的函数重载而可能产生意想不到的 CE 或 WA。**原则：** `int` 够用就别 `long long`。

🔴 **ios::sync\_with\_stdio(false) 缺失 (12.5%)：** 这是一个巨大的性能陷阱。你64.9%的代码使用 `cin/cout`，却很少关闭C流的同步。在遇到  $10^5$  级别的大数据输入时，你的I/O时间将比使用 `scanf/printf` 慢10倍以上。这在比赛中是致命的TLE原因。**立即养成习惯：** 在每个 `main` 函数开头加上 `ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);`。

🔴 **代码长度中位数55行：** 这表明你的代码过于“简单粗暴”，对于复杂问题，55行往往意味着逻辑耦合、缺少模块化。为了攻克省选难题，你需要学习如何写出**结构化、可读性更强的代码**。例如，将DFS、DP、图论建图等逻辑封装成独立的函数。

🔴 **变量名过度缩写：** `l` , `r` , `u` , `v` , `n` , `m` , `x` , `y` , `a` , `b` , `c` ... 这些是常见的，但当变量作为函数参数或全局变量时，非常难以阅读和调试。**建议：** 对于超过3个字母的变量，使用有意义的英文单词或驼峰命名，例如 `edgeCount` , `currentNode` , `answer`。

## 【定制训练题单（6个月路线图）】

这6个月的目标非常明确：**从普及组选手蜕变为省选入门选手。**

### 第一阶段（Month 1-2）：夯实基础，补齐短板，冲击S组入门

- 目标：补齐 保险系数最大 的考点。

- **1. 图论基础模型 (MST)**
  - 知识点: Prim, Kruskal
  - 题目: P3366 (模板), P1546 (USACO 最短网络)
- **2. 动态规划突破 (树形 & 区间)**
  - 知识点: 树上背包、区间合并
  - 题目: P2014 (选课-树形背包), P1880 (石子合并-区间DP)
- **3. 数学武器库 (快速幂 & 逆元 & 组合数)**
  - 知识点: 逆元、组合数计算
  - 题目: P3811 (乘法逆元), P3375 (快速幂), P1026 (统计单词个数 - 练习DP+字符串)

## 第二阶段 (Month 3-4) : 数据结构与算法突破, 掌握核心模型

- **目标:** 具备独立解决中等难度 S 组题目的能力。
- **4. 字符串算法 (KMP & AC自动机)**
  - 知识点: KMP + 自动机思想
  - 题目: P3375 (KMP模板), P3796 (AC自动机模板)
- **5. 图论高级模型 (二分图 & 差分约束)**
  - 知识点: 匈牙利算法、差分约束系统
  - 题目: P3386 (二分图最大匹配), P2294 (等式/不等式)
- **6. 复杂 DP (状压 & 可持久化 & 单调队列优化)**
  - 知识点: 状态压缩DP、单调队列优化DP
  - 题目: P1433 (吃奶酪-状压), P2698 (花盆-单调队列)

## 第三阶段 (Month 5-6) : 提速与稳定, 形成省选思维

- **目标:** 提升解题速度, 初步接触省选思维。
- **7. 高级图论 (强连通分量 & 缩点 & 欧拉图)**
  - 知识点: Tarjan, 缩点建新图, 欧拉回路判定
  - 题目: P2341 (受欢迎的牛), P1341 (无序字母对)
- **8. 综合练习与模拟赛**
  - 每周完成2-3场CSP-S模拟赛, 严格控制时间。重点练习**如何拿到部分分**, 例如, 对于 P3304, 先思考暴力如何写。
- **9. 复盘沉淀 (最重要的)**
  - 每次卡题后, 必须进行**四段式复盘**:  
**赛时模型:** 我最初是怎么想的? 建了什么模?  
**错因:** 哪里想错了? 是算法用错? 边界没判? 还是代码实现有bug?  
**正解性质:** 正解利用了题目的什么特殊性质? (例如, 树的直径的“必经边”)  
**代码不变量:** 我的代码逻辑是否正确? pushdown的lazy标记是否完全正确传递?

## 【核心建议（优先级排序）】

- **紧急: 补加IO优化:** 在你的 `main` 函数第一件事就是 `ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);`。这能瞬间解决你50%的大数据TLE问题。
- **紧急: 系统学习树形DP & 区间DP:** 这是通往省选的第一扇门, 也你目前最致命的软肋。必须花一周时间专门攻克。
- **重要: 学习gdb调试:** 放弃“靠猜”的调试方式。学会使用 `gdb` 打断点, 查看变量值, 单步跟踪, 能让你在5分钟内解决之前需要50次提交才能解决的bug。
- **重要: 建立错题本:** 分析你的死磕题目TOP, 你会发现都是卡在“模型抽象”上。建立一个错题本, 记下每次卡题时你错误的思维模型和正确的模型。
- **重要: 改掉 `#define int long long` 的坏习惯:** 只在必要时使用。这能提升你的代码质量和性能。
- **建议: 每周打一场 Codeforces 或 AtCoder:** 体验不同的赛制和难度曲线, 特别是CF的Div.2, 可以有效锻炼你在高压下的快速反应能力。
- **建议: 开始刷省选/NOI难度的题目:** 不用怕, 从5道题开始, 哪怕是只看题解。目的是开阔视野, 知道“原来还有这种操作”。

## 【未通过题目专属题解（从暴力到正解）】

以下将对你的5道卡题进行深度剖析, 形塑你的解题思维。

### 题目: P5304 [GXOI/GZOI2019] 旅行者

- a) **AI 题解摘要:** 这是一道 **多源最短路** 的变种问题, 核心是求**若干关键点**之间的最短距离。关键技巧是**二进制分组**, 将多个点对的最短路问题转化为 $O(\log N)$ 次单源最短路问题。
- b) **暴力思路:** \* **思路一 (朴素多源暴力):** 对每个关键点都跑一次Dijkstra, 记录它到其他所有点的距离, 然后取所有关键点两两之间的最小值。\* **复杂度:**  $O(K * (M \log N))$ 。当K很大 (如K接近N) 时, 时间复杂度爆炸。\* **部分分:** 能拿到当K很小或图很特殊时的分数。
- c) **瓶颈在哪里?** \* 你重复跑了K次单源最短路, 但最终只需要所有关键点对之间的最小值。这里面有大量的冗余计算。
- d) **关键性质/不变量观察:** **Key Idea:** 假设答案的两个关键点为 `u` 和 `v`。它们拥有不同的二进制表示 (只要不是同一个点)。因此, 至少存在一位二进制位, 使得 `u` 在这一位上是0, 而 `v` 是1。
- e) **最终正解的推导与核心代码结构:** 1. **分层 (二进制分组):** 枚举二进制位 `i` ( $0 \leq i \leq 20$ )。2. **建立超级源点/汇点:** \* 对于每个关键点 `p`: \* 如果 `p` 的第 `i` 位是 1, 那么从超级源点 `S` 向 `p` 连一条长度为 `0` 的边。\* 如果 `p` 的第 `i` 位是 0, 那么从 `p` 向超级汇点 `T` 连一条长度为 `0` 的边。3. **跑单源最短路:** 用 Dijkstra 算法, 从 `S` 出发, 计算到所有点的最短路, 最终 `dist[T]` 即为这一轮分组下找到的最短路候选值。4. **交换角色:** 将源点与汇点的角色互换 (即

第*i*位为0的点连源点，为1的点连汇点），再跑一次 Dijkstra，更新答案。5. **取最小值**：对于所有位，我们都能得到至少一个候选值，取所有候选值的最小值即为答案。

**核心代码片段 (伪代码)：**

```
for (int bit = 0; (1<<bit) <= K; ++bit) {
    // 超级源点 S = n + 1, 超级汇点 T = n + 2
    // 清空边, 重新建图
    build_special_edges(bit, 0); // 1连S, 0连T
    ans = min(ans, dijkstra(S, T));

    build_special_edges(bit, 1); // 1连T, 0连S
    ans = min(ans, dijkstra(S, T));
}
```

f) **推荐同类题**：\* **P1629 [邮递员送信]**：多源单源最短路的入门题，练习建立超级源点的思想。  
\* **P1462 [通往奥格瑞玛的道路]**：二分答案+单源最短路，组合拳。

## 题目：P2471 [SCOI2007] 降雨量

a) **AI 题解摘要**：这道题的难点不在于算法，而在于**逻辑分类讨论**的完备性。需要严格判断年份是否未知、降水量的大小关系、以及是否假。

b) **暴力思路**：\* 直接模拟。对于每个询问  $(X, Y)$ ，遍历  $X$  到  $Y$  之间所有已知年份的降水量，然后根据题目条件进行判断。\* **复杂度**： $O(NQ)$ ，显然无法通过大数据。\* **部分分**：可以拿到数据量较小时的部分分数。

c) **瓶颈在哪里**？\* 查询区间内最大值。这是典型的**RMQ (区间最值查询)**问题。

d) **关键性质/不变量观察**：\* 年份是严格递增的，这为使用**二分查找**定位年份提供了基础。\* **MUST TRUE** 的条件是三个最严格的： $X$  年已知， $Y$  年已知， $(X, Y)$  之间年份全部已知，且  $\text{rain}[X] \geq \text{rain}[Y]$ ，且中间所有年份的降水量  $< \text{rain}[Y]$ 。\* **FALSE** 是最容易判定的。\* 剩下的情况都是 **MAYBE**。

e) **最终正解的推导与核心代码结构**：1. **数据结构**：用 `map<int, int>` 存储已知年份到降水量的映射。另外用两个数组分别存储年份和降水量，方便进行二分和RMQ。2. **预处理**：使用**ST表**或**线段树**预处理区间最大值，实现  $O(1)$  查询。3. **查询逻辑 (核心)**：\* `pos_x = lower_bound(years, X)`; `pos_y = upper_bound(years, Y) - 1`; // 找到Y年或之前的第一个已知年份  
\* **情况1 (X已知, Y已知)**：\* `if (rain[X] < rain[Y])` -> 直接输出 **FALSE**。\* 查询  $(X, Y)$  之间的最大值 `max_mid` (不包含两 endpoints)。\* `if (max_mid >= rain[Y])` -> **FALSE**。\* 如果 `pos_y - pos_x == Y - X` (说明中间年份全部已知) -> **TRUE**。\* 否则 -> **MAYBE**。\* **情况2 (X未知, Y已知)**：\* 查询 `pos_x` 到 `pos_y` 之间的最大值 `max_mid`。\* `if (max_mid >= rain[Y])` ->



FALSE。\* 否则 -> MAYBE。\* **情况3 (X已知, Y未知):** \* 查询 pos\_x 到 pos\_y 之间的最大值 max\_mid。\* if (max\_mid >= rain[X]) -> FALSE。\* 否则 -> MAYBE。\* **情况4 (都未知):** 直接 MAYBE。

**f) 推荐同类题:** \* P2286 [HNOI2004] 宠物收养场: 同样是二分、分类讨论和STL的运用, 锻炼逻辑严谨性。

## 题目: P5888 传球游戏

**a) AI 题解摘要:** 这是一个**计数DP**问题, 但状态数高达  $O(NK)$ , 不可行。核心观察是, **只有极少数人是“特殊”的** (即被“不能传给”关系影响), 其余人的转移是相同的。利用**矩阵快速幂或快速幂优化**来处理这些“普通”人的转移。

**b) 暴力思路:** \* dp[time][person] 表示经过 time 次传球后, 球在 person 手上的方案数。\* 状态转移: dp[time+1][j] += dp[time][i], 其中 i 和 j 可以传球。\* **复杂度:**  $O(K * N^2)$  或  $O(K * (N+M))$ 。N 和 K 都很大时, 不可行。\* **部分分:** 可以拿到 N 小 或 K 小的部分分。

**c) 瓶颈在哪里?** \* N 和 K 都很大, 无法枚举所有人和所有时间步。

**d) 关键性质/不变量观察:** \* M最多只有 $5 \cdot 10^5$ , 但限制只涉及最多2M个人。\* 因此, 大部分人是“普通”的: 他们只被“不能传给”某一个人, 而这个人也是普通人。\* 对于 n 个普通人, 他们的转移是**完全对称的**。从任意一个普通人出发, 下一轮可以传给: 除了他自己以外的其他 n-1 个普通人, 以及 m 个“特殊”人。

**e) 最终正解的推导与核心代码结构:** 1. **压点:** \* **特殊人 (记为 S):** 被限制关系影响的所有人, 最多 2M 个。\* **普通人 (记为 N\_normal):**  $N - |S|$  个。2. **状态:** dp[time][0] 表示球在特殊人手上的方案数 (由于特殊人之间转移不同, 需要分开算), dp[time][1] 表示球在任何等价的普通人手上的方案数。3. **转移 (针对普通人):** \*  $next\_dp[1] = dp[1] * (N\_normal - 1) + dp[0] * (某值)$ 。具体公式需要分类讨论, 核心是利用对称性, 将状态压缩到2个。4. **优化:** 直接按时间步 K 转移是  $O(K)$  的。但我们只需要知道最后的方案数, 可以将**每一步的转移写成线性递推形式**, 然后用**矩阵快速幂**在  $O(\log K)$  时间内求出最终答案。矩阵大小为  $(|S|+1) * (|S|+1)$ 。5. **核心:** 普通人的自洽: 对于一个普通人, 他下一次可以选择传给任意一个其他普通人和所有特殊人。所以转移是“自己”+“别人”。

**f) 推荐同类题:** \* P3758 [TJOI2010] 电影之矩阵加速DP: 经典的使用矩阵快速幂优化DP计数。\* P1962 斐波那契数列: 矩阵快速幂的入门, 理解如何将线性递推转化为矩阵乘法。

## 题目: P3304 [SDOI2013] 直径

**a) AI 题解摘要:** 树的直径的经典问题变种。关键在于运用**必要性证明**的思维。直径上的边被所有直径经过, 当且仅当它是所有直径的“公共部分”。用两次DFS求出直径后, 从两端出发, 看哪些边是“必须被经过”的。

**b) 暴力思路：** \* 枚举所有点对，求最远距离。然后**求出所有直径**，再求它们的公共边。  $O(N^3)$  都难写对。

**c) 瓶颈在哪里？** \* 如何高效地求出所有直径，并判断公共边。

**d) 关键性质/不变量观察：** \* **性质1：**树的直径不一定唯一，但所有直径的中点是同一个点（或一条边）。 \* **性质2：**所有直径的**公共部分**是一条**连续**的路径（即“直径的核”）。 \* **性质3：**设我们第一次DFS找到直径的一个端点为  $L$ ，另一个端点为  $R$ 。那么，**对于直径上的任意一条边  $e$** ，如果存在至少一条直径不经过  $e$ ，那么  $e$  一定不在所有直径的公共部分中。

**e) 最终正解的推导与核心代码结构：** 1. **求直径端点：**两次DFS(BFS)，求出直径长度和端点  $L$ ， $R$ 。 2. **化为序列：**将直径上的路径（从 $L$ 到 $R$ ）拉成一个序列，这样问题变成了**在一个序列上，求一个最长的连续子段，使得该子段上的每条边都被所有直径包含**。 3. **从左边缩圈（从左端点 $L$ 出发）：** \* 从直径的端点  $L$  开始，沿着直径向  $R$  走，直到遇到一个点  $u$ ，使得  $u$  节点有一棵**不经过直径**的子树，且这个子树的深度等于  $u$  到  $L$  的距离。这说明，以  $L$  为端点的直径，可以“**绕路**”到这条子树，所以从 $L$ 到 $u$ 这一段路径，并不被所有直径包含。 \* 不断向右逼近，找到最靠右的“不可绕路”的点  $L\_fixed$ 。 4. **从右边缩圈（从右端点 $R$ 出发）：** \* 同样的逻辑，从 $R$ 向左逼近，找到最靠左的点  $R\_fixed$ 。 5. **答案：**  $L\_fixed$  与  $R\_fixed$  之间相隔的边数，就是所有直径的公共边数量。这条路径就是所有直径的“核”。

**举例如下：** 假设直径序列为  $1-2-3-4-5-6$  ( $L=1, R=6$ )。 \* 从  $1$  出发，发现节点  $2$  有一条深度为  $1$  的侧链，意味着存在一条直径  $2-\dots-6$ ，绕过了边  $1-2$ 。那么  $L\_fixed$  至少是  $2$ 。 \* 从  $6$  出发，发现节点  $5$  的侧链深度为  $1$ ，那么  $R\_fixed$  至少是  $5$ 。 \* 那么公共边是  $2-3, 3-4, 4-5$ ，共3条。

**f) 推荐同类题：** \* **P1073 [NOIP2009] 最优贸易：**虽然不是树的直径，但同样是图论中的“路径分离”，以及利用关键路径进行性质推导。

## 题目：B4254 [科大国创杯小学组 2024] 博弈

**a) AI 题解摘要：**这是一道**博弈论**题，基本模型是一个**公平组合游戏**。你需要把整个棋盘的状态压缩成一个**SG函数**值，然后判断先手胜负。对于这种网格型博弈，状态数通常极大，需要寻找规律或用状压DP计算SG值。

**b) 暴力思路：** \* **暴力搜索：**直接递归（记忆化搜索）所有可能的棋盘状态。棋盘大小为  $3*3$  或  $4*4$ ，状态数  $3^9$  或  $3^{16}$ 。可枚举。 \* 对于  $n*n$  的棋盘 ( $n \geq 5$ )，状态数爆炸。

**c) 瓶颈在哪里？** \* 状态空间过大。对于  $n=10$ ，状态数是天文数字。无法直接暴力。

**d) 关键性质/不变量观察：** \* **博弈的公平组合性：**这是一个典型的“Impartial Combinatorial Game”。每个棋盘状态可以用一个非负整数 (SG值) 来表征。 \* **棋盘，棋子，必胜/必败：**由于这个题目是一个“小学组”题目（可能，不是真的小学组），且  $n$  不大，其状态可能可以用**状压表**

示，然后DP出SG值。\* 更可能的是，这道题**存在一个数学结论**：比如**后手必胜**当且仅当棋盘是某个特定形状（如通过了某个对称操作后不变），或者**先手必胜**当且仅当  $n$  为奇数等等。

**e) 最终正解的推导与核心代码结构：**

- \* **第一步：猜结论。**试一下  $n=1, n=2, n=3$  的SG值。
- \* **第二步：找规律。**看SG(n) 是否等于 0 (后手胜) 或非0 (先手胜)。
- \* **第三步：如果没有结论（即题目需要计算SG值）**，由于  $n \leq 20$ ，但状态数是  $3^{400}$ ，不可行。但棋盘可以做**极小化的必胜策略**，题解通常是：**将黑色棋子的分布在对称性上做文章**。例如答案可能是  $n$  模某个数的结果。
- \* **第四步：核心代码。**因为没有比赛原题代码，无法给出具体代码。但如果是状压+SG，核心是：  

```
cpp // 假设状态用mask表示 int sg[mask]; unordered_set<int> mex_set; for (每个合法走法) {
int new_mask = apply_move(mask); mex_set.insert(sg[new_mask]); } // 找mex
```

\* **为你提供的思路：**对于这种“小学组”博弈题，**不要走弯路**。直接去**手算小规模样例**，猜测必胜/必败的数学条件。比如：

- \* 如果棋盘有唯一对称轴，后手可以一直模仿先手操作，那么后手必胜。
- \* 否则，先手在对称轴上下一子，破坏了对称性，之后继续模仿后手。

**f) 推荐同类题：**

- \* **P2197 [USACO08JAN] Cow Checkers**：经典的 Nim 博弈 变形。
- \* **P4363 [九省联考2018] 一双木棋**：一道经典的**轮廓线DP**+博弈论，学习如何用 SG 值和状压解决棋盘博弈问题。